

Модуль 6. Введение в брокеры сообщений и паттерн Producer/Consumer

6.1. Зачем мы здесь: от синхронного API к асинхронной обработке

В предыдущих модулях мы построили связку из трёх сервисов: API на FastAPI, база данных PostgreSQL и брокер Redis. Но пока что мы использовали Redis лишь как хранилище для метаданных очереди (`XLEN` , `XADD`). API всё ещё делает всю работу **синхронно**: когда пользователь отправляет запрос, API немедленно выполняет все действия — пишет в БД, кладёт задачу в Redis, отвечает клиенту.

Теперь нам нужно сделать качественный скачок. Мы должны понять, **почему** некоторые задачи нельзя выполнять внутри HTTP-запроса, и как **посредник** (брокер сообщений) решает эту проблему. Этот модуль — чисто теоретический с практической симуляцией. Мы не будем использовать Docker, Celery или Kafka. Мы построим модель очереди на чистом Python, чтобы увидеть, как работает паттерн изнутри.

6.2. Проблема синхронности: почему API не должен ждать

6.2.1. Что происходит внутри HTTP-запроса

Когда пользователь открывает ваш сайт и нажимает кнопку «Обучить модель», его браузер отправляет **HTTP-запрос** на ваш сервер. В этот момент открывается **соединение** между браузером и сервером. Сервер должен обработать запрос и вернуть ответ. Пока ответ не получен, браузер показывает пользователю крутящийся спиннер или пустой экран.

Аналогия: звонок в службу поддержки

Представьте, что вы звоните в банк. Оператор берёт трубку и говорит: «Да, конечно, я сейчас оформлю вам кредит. Подождите на линии». И вы сидите в трубке 45 минут, слушая музыку удержания. Вы не можете повесить трубку, потому что иначе оператор не сможет вам перезвонить с результатом. Вы не можете позвонить другу, потому что линия занята. Вы просто ждёте.

Это и есть **синхронная обработка**. Пользователь «сидит на линии», пока сервер делает работу.

6.2.2. Быстрые vs медленные операции

Разделим всё, что может делать API, на два типа:

Тип	Примеры	Время	Где выполнять
Быстрые (синхронные)	Проверка логина, чтение из БД, простое предсказание моделью	10–200 мс	Прямо в HTTP-запросе
Медленные (асинхронные)	Обучение модели на 10 ГБ данных, генерация отчёта в PDF, рассылка 10 000 писем, обработка видео	30 сек – несколько часов	Нельзя выполнять в HTTP-запросе

Почему нельзя выполнять медленные задачи в запросе?

- 1. Таймаут соединения.** Браузер, балансировщик нагрузки, облачный шлюз — все они имеют таймаут. Обычно 30–60 секунд. Если задача длится дольше — соединение рвётся, пользователь видит ошибку.
- 2. Исчерпание ресурсов.** Веб-сервер (Uvicorn/Gunicorn) имеет пул рабочих процессов. Если 10 пользователей одновременно запустили обучение модели — все 10 процессов заняты на час. 11-й пользователь не сможет даже открыть главную страницу. Сайт «лёг».
- 3. Плохой пользовательский опыт.** Никто не хочет смотреть на крутящийся спиннер 10 минут. Пользователь хочет: нажал кнопку, получил подтверждение «Заказ принят», и ушёл пить кофе.
- 4. Ненадёжность.** Если сервер упал во время выполнения задачи — задача потеряна. Пользователь не знает, выполнилась она или нет.

6.2.3. Аналогия: ресторан быстрого питания

Представьте McDonald's. Вы подходите к кассе и заказываете:

- Бургер (готовится 2 минуты).
- Кофе (готовится 30 секунд).
- Индивидуальный торт на заказ (готовится 3 часа, потому что кондитер должен вымесить тесто, выпечь, оформить).

Что будет, если кассир станет ждать, пока кондитер сделает торт?

- Касса заблокирована 3 часа.
- Очередь вырастает до улицы.
- Люди, хотевшие только кофе, уходят.
- Ресторан терпит убытки.

Правильное решение: Кассир принимает заказ на торт, даёт вам номерок (например, №42), и сразу обслуживает следующего клиента. Кондитер работает в отдельной комнате. Когда торт готов — он кричит: «Номерок 42!» Вы подходите и забираете.

В этом сценарии:

- **Кассир** = API (Producer).
- **Кондитер** = Фоновый воркер (Consumer).
- **Номерок** = ID задачи, который API возвращает пользователю.
- **Очередь заказов на торты** = Брокер сообщений.

6.3. Паттерн Producer/Consumer: разделение ответственности

6.3.1. Три актёра системы

Паттерн **Producer/Consumer** (Производитель/Потребитель) — это способ организации работы, при котором один компонент создаёт задачи, другой их выполняет, а между ними стоит **посредник** (очередь).

Актёр 1: Producer (Производитель, отправитель)

Producer — это тот, кто **создаёт сообщение** и кладёт его в очередь. В нашем контексте это обычно API. Producer не знает, кто и когда будет обрабатывать сообщение. Его работа заканчивается тем, что сообщение оказалось в очереди.

Аналогия: Официант в ресторане. Он подходит к столику, записывает заказ, несёт бумажку на кухню и кладёт в стопку. Он не ждёт, пока повар приготовит блюдо. Он сразу идёт обслуживать следующий столик.

Актёр 2: Queue (Очередь, брокер)

Queue — это **посредник**, хранилище сообщений. Она принимает сообщения от Producer'ов и отдаёт их Consumer'ам. Очередь гарантирует, что сообщение не потеряется, пока его не обработают.

Аналогия: Стопка заказов на кухне. Она стоит между залом и поварами. Если повар занят — заказы накапливаются в стопке. Если официантов много — стопка растёт, но заказы не пропадают.

Актёр 3: Consumer (Потребитель, исполнитель)

Consumer — это тот, кто **забирает сообщение** из очереди и выполняет работу. Он может работать медленно, может падать и перезапускаться, может быть одним или десятью — очередь это скрывает от Producer'a.

Аналогия: Повар на кухне. Он берёт верхний заказ из стопки, готовит блюдо, кричит «Забирайте!», и берёт следующий заказ. Если повар один — готовит по очереди. Если поваров три — делят стопку между собой.

6.3.2. Почему это разделение мощное

1. Независимость скоростей

Producer работает быстро (принял заказ — 50 мс). Consumer работает медленно (готовил блюдо — 10 мин). Они не ждут друг друга. Очередь сглаживает разницу в скоростях.

2. Масштабируемость

Если заказов стало много — вы не нанимаете больше официантов (Producer'ов), потому что они и так быстрые. Вы нанимаете больше поваров (Consumer'ов). Очередь позволяет добавлять Consumer'ов без изменения кода Producer'a.

3. Отказоустойчивость

Если повар (Consumer) заболел и ушёл — заказы не пропадают. Они лежат в стопке (очереди). Когда повар вернётся или придёт новый — он продолжит с того места, где остановился.

4. Гибкость

Producer написан на Python (FastAPI). Consumer может быть написан на Python, Go, Java — очередь не различает языки. Главное, чтобы все понимали формат сообщения.

6.4. Очередь как буфер: выравнивание нагрузки

6.4.1. Что такое «буфер»

Буфер — это промежуточная ёмкость, которая принимает поток данных с одной скоростью и отдаёт с другой.

Аналогия: ванна с водой

Представьте ванну. Сверху льётся вода из крана (Producer). Снизу вода уходит в слив (Consumer). Если кран льёт быстрее, чем слив убирает — ванна наполняется. Если слив работает быстрее — ванна опустошается. Ванна **выравнивает** потоки: кратковременные всплески от крана не затопляют пол, потому что ванна их поглощает.

Очередь — это ванна для задач. Если на API пришёл всплеск запросов (например, утром в понедельник все загружают отчёты) — очередь накапливает задачи. Consumer'ы работают с постоянной скоростью и постепенно разбирают накопившееся.

6.4.2. Защита от перегрузки (Backpressure)

Что произойдёт, если кран льёт слишком долго, а слив засорился? Ванна переполнится.

В системах сообщений есть два подхода:

1. **Очередь с ограничением.** Когда очередь достигает максимального размера — новые сообщения отклоняются. Producer получает ошибку и может сообщить пользователю: «Система перегружена, попробуйте позже».
2. **Очередь без ограничения.** Очередь растёт, пока не закончится память. Это опасно, но иногда применяется с мониторингом.

Аналогия: В ресторане, когда стопка заказов достигает потолка, менеджер вешает табличку «Стоп-лист: кухня не принимает новые сложные заказы». Официанты (Producer'ы) предупреждают гостей: «Извините, торты сегодня не заказываем, кухня завалена».

6.5. Гарантии доставки: at-most-once, at-least-once, exactly-once

Это один из самых важных концептов в мире очередей. Он определяет, **сколько раз** сообщение будет обработано. Разберём на бытовых примерах.

6.5.1. At-most-once (Не более одного раза)

Определение: Сообщение будет доставлено **ноль или один раз**. Оно может потеряться, но гарантированно не будет обработано дважды.

Аналогия: SMS без подтверждения

Вы отправляете другу SMS: «Забери меня в 8». Вы не знаете, дошло ли сообщение. Может, дошло — друг вас заберёт. Может, не дошло — друг не приедет, и вы останетесь ждать. Но вы точно знаете: друг не приедет дважды, потому что SMS не продублировалось.

Когда используется: Логирование, метрики, потоковая аналитика, где потеря нескольких сообщений не критична. Самая высокая производительность, но низкая надёжность.

6.5.2. At-least-once (Не менее одного раза)

Определение: Сообщение будет доставлено **один или более раз**. Оно гарантированно не потеряется, но может прийти дважды (или трижды) при сбоях.

Аналогия: Письмо с уведомлением о вручении

Вы отправляете заказное письмо. Почта гарантирует, что письмо дойдёт. Но из-за сбоя в сортировке письмо может попасть к адресату дважды. Получатель откроет первое письмо, оплатит счёт. Потом получит второе такое же письмо. Если он не проверит, что уже оплатил — оплатит ещё раз.

Проблема: Дублирование. Consumer может обработать одно и то же сообщение дважды.

Решение: Consumer должен быть **идемпотентным** — повторная обработка не меняет результата. Мы подробно разберём идемпотентность в Модуле 12.

Когда используется: Самый распространённый режим. Email-рассылки, обработка заказов, индексация документов.

6.5.3. Exactly-once (Ровно один раз)

Определение: Сообщение будет доставлено и обработано **строго один раз**. Ни потерь, ни дублей.

Аналогия: Банковский перевод

Вы переводите 1000 рублей другу. Банк гарантирует: деньги спишутся с вашего счёта ровно один раз и зачислятся другу ровно один раз. Не ноль раз (потеря), не два раза (двойное списание).

Сложность: Это самая дорогая гарантия. Требуется сложной координации между брокером и Consumer'ом: транзакции, двухфазный коммит, дедупликация.

Когда используется: Финансовые операции, управление запасами, критичные транзакции.

6.5.4. Сравнительная таблица

Гарантия	Потеря	Дубли	Сложность	Пример использования
At-most-once	Возможна	Нет	Низкая	Логи, метрики
At-least-once	Нет	Возможны	Средняя	Рассылка писем, ML-инференс
Exactly-once	Нет	Нет	Высокая	Платежи, бронирование

6.6. Практика: моделируем очередь заказов в кафе на Python

Теперь мы напишем простую симуляцию на чистом Python. Это не production-код, а обучающая модель, которая покажет, как взаимодействуют Producer, Queue и Consumer.

Шаг 1. Создаём файл симуляции

В папке `~/docker-module5` (или новой `~/docker-module6`) создайте файл `cafe_simulation.py` .

```
In [1]: import time
import random
```

```

import threading
from queue import Queue, Empty
from datetime import datetime

# =====
# КЛАСС: Очередь заказов (имитация брокера сообщений)
# =====

class OrderQueue:
    """
    Это наш брокер сообщений.
    В реальности это был бы Redis, RabbitMQ или Kafka.
    Здесь мы используем потокобезопасную очередь из Python.
    """

    def __init__(self, maxsize=10):
        self._queue = Queue(maxsize=maxsize)
        self.closed = False

    def put(self, order):
        """Producer кладёт заказ в очередь."""
        if self.closed:
            raise Exception("Очередь закрыта!")
        try:
            self._queue.put(order, block=False)
            print(f"[БРОКЕР] Заказ #{order['id']} принят в очередь. Размер очереди: {self._queue.qsize()}")
            return True
        except:
            print(f"[БРОКЕР] ОЧЕРЕДЬ ПЕРЕПОЛНЕНА! Заказ #{order['id']} отклонён.")
            return False

    def get(self, timeout=1):
        """Consumer забирает заказ из очереди."""
        try:
            return self._queue.get(timeout=timeout)
        except Empty:
            return None

    def task_done(self):
        """Consumer сообщает, что заказ обработан."""
        self._queue.task_done()

    def close(self):
        self.closed = True

```

```

# =====
# КЛАСС: Официант (Producer)
# =====

class Waiter(threading.Thread):
    """
    Официант – это наш API.
    Он быстро принимает заказы и кладёт их в очередь.
    Не ждёт, пока повар приготовит.
    """

    def __init__(self, name, queue, num_orders=20):
        super().__init__()
        self.name = name
        self.queue = queue
        self.num_orders = num_orders
        self.order_id = 0

    def run(self):
        print(f"[{self.name}] Начал смену. Будет принято {self.num_orders} заказов.")

        for _ in range(self.num_orders):
            self.order_id += 1
            order = {
                "id": self.order_id,
                "dish": random.choice(["Борщ", "Стейк", "Салат", "Паста", "Суп"]),
                "complexity": random.randint(1, 5), # Сложность от 1 до 5 секунд
                "table": random.randint(1, 10),
                "timestamp": datetime.now().isoformat()
            }

            # Официант быстро пишет заказ и несёт на кухню
            success = self.queue.put(order)

            if success:
                # Возвращает клиенту "номерок"
                print(f"[{self.name}] Клиенту за столом #{order['table']} выдан номерок #{order['id']}")

            # Официант обходит зал, пауза между заказами
            time.sleep(random.uniform(0.1, 0.5))

        print(f"[{self.name}] Смена окончена. Все заказы приняты.")

# =====
# КЛАСС: Повар (Consumer)

```



```

# =====
class Cook(threading.Thread):
    """
    Повар – это наш воркер (worker).
    Он работает медленно, берёт заказы из очереди и готовит.
    """
    def __init__(self, name, queue):
        super().__init__()
        self.name = name
        self.queue = queue
        self.orders_cooked = 0
        self.daemon = True # Поток-демон: завершится вместе с программой

    def run(self):
        print(f"[{self.name}] Выходит на кухню.")

        while True:
            # Повар смотрит в очередь (ждёт до 2 секунд)
            order = self.queue.get(timeout=2)

            if order is None:
                # Если очередь пуста 2 секунды – повар отдыхает
                if self.queue.closed:
                    print(f"[{self.name}] Кухня закрыта. Уходит домой.")
                    break
                continue

            # Готовит блюдо (время = сложность)
            print(f"[{self.name}] Начал готовить заказ #{order['id']}: {order['dish']} (сложность {order['complexity']})")
            time.sleep(order['complexity'])

            # Блюдо готово
            print(f"[{self.name}] ЗАКАЗ ГОТОВ! #{order['id']}: {order['dish']} для стола #{order['table']}")
            self.orders_cooked += 1
            self.queue.task_done()

        print(f"[{self.name}] Приготовил всего: {self.orders_cooked} блюд.")

# =====
# ГЛАВНАЯ ФУНКЦИЯ: запускаем ресторан
# =====
def main():
    print("=" * 60)

```

```
print("СИМУЛЯЦИЯ: РЕСТОРАН С ОЧЕРЕДЬЮ ЗАКАЗОВ")
print("=" * 60)
print()
print("Актёры:")
print("  - Официант (Producer) = API, принимает запросы")
print("  - Очередь (Queue)       = Брокер сообщений")
print("  - Повара (Consumers)     = Воркеры, выполняют задачи")
print()

# Создаём брокер (очередь на 10 заказов)
order_queue = OrderQueue(maxsize=10)

# Создаём официанта (1 Producer)
waiter = Waiter(name="Официант Петя", queue=order_queue, num_orders=15)

# Создаём поваров (2 Consumers)
cook1 = Cook(name="Повар Вася", queue=order_queue)
cook2 = Cook(name="Повар Гриша", queue=order_queue)

# Запускаем поваров (они ждут заказы)
cook1.start()
cook2.start()

# Запускаем официанта (он начинает приносить заказы)
waiter.start()

# Ждём, пока официант закончит
waiter.join()
print()
print("[МЕНЕДЖЕР] Официант закончил. Ждём, пока кухня разберёт остатки...")

# Ждём, пока очередь опустеет
order_queue._queue.join()
order_queue.close()

# Даём поварам время увидеть, что очередь закрыта
time.sleep(3)

print()
print("=" * 60)
print("ИТОГИ СМЕНЫ:")
print(f"  Всего заказов принято: {waiter.order_id}")
print(f"  {cook1.name} приготовил: {cook1.orders_cooked}")
print(f"  {cook2.name} приготовил: {cook2.orders_cooked}")
print(f"  Очередь разобрана полностью: {order_queue._queue.empty()}")
print("=" * 60)
```

```
if __name__ == "__main__":  
    main()
```

Шаг 2. Запускаем и наблюдаем

```
In [2]: python cafe_simulation.py
```

Что вы увидите:

1. Официант (Producer) быстро принимает заказы и кладёт их в очередь.
2. Когда очередь заполняется (10 заказов) — новые заказы отклоняются (backpressure).
3. Два повара (Consumers) берут заказы из очереди и готовят.
4. Повара работают параллельно: пока один готовит борщ 5 секунд, другой делает салат 1 секунду.
5. Официант заканчивает работу раньше, чем повара. Он не ждал!
6. Программа ждёт, пока очередь полностью разберётся.

Шаг 3. Эксперименты

Эксперимент 1. Один повар вместо двух Закомментируйте `cook2.start()`. Вы увидите, что очередь быстро переполняется, потому что один повар не справляется с потоком заказов.

Эксперимент 2. Очередь без лимита Измените `maxsize=10` на `maxsize=0` (0 означает бесконечность). Очередь никогда не отклонит заказ, но будет расти в памяти.

Эксперимент 3. Медленный официант Измените `time.sleep(random.uniform(0.1, 0.5))` на `time.sleep(2)`. Официант будет принимать заказы реже, и поварам будет легче справляться.

6.7. Связь с реальными технологиями

Наша симуляция — это упрощённая модель. В реальности роли распределяются так:

Наша симуляция	Реальная технология	Роль
<code>OrderQueue</code> (Python Queue)	Redis , RabbitMQ, Kafka	Брокер сообщений
<code>Waiter</code>	FastAPI/Flask endpoint	Producer

Наша симуляция	Реальная технология	Роль
<code>Cook</code>	Celery Worker , или Kafka Consumer	Consumer
<code>order['id']</code>	message_id , task_id	Идентификатор задачи
<code>queue.put()</code>	<code>celery.send_task()</code> , <code>redis.xadd()</code>	Отправка сообщения
<code>queue.get()</code>	<code>celery worker</code> , <code>redis.xread()</code>	Получение сообщения
<code>task_done()</code>	ACK (acknowledgment)	Подтверждение обработки

6.8. Итоги модуля: чек-лист

- ☐ Понимаю, почему медленные задачи (обучение, отчёты) нельзя выполнять внутри HTTP-запроса.
- ☐ Знаю три проблемы синхронности: таймауты, исчерпание ресурсов, плохой UX.
- ☐ Понимаю паттерн **Producer/Consumer** и роль каждого актёра.
- ☐ Знаю, что **Producer** создаёт сообщение и кладёт его в очередь, не дожидаясь результата.
- ☐ Знаю, что **Queue (брокер)** — это посредник, хранилище и буфер между Producer и Consumer.
- ☐ Знаю, что **Consumer** забирает сообщение из очереди и выполняет работу.
- ☐ Понимаю, как очередь **выравнивает нагрузку** и защищает от всплесков (backpressure).
- ☐ Знаю три **гарантии доставки**: at-most-once, at-least-once, exactly-once.
- ☐ Могу объяснить каждую гарантию на бытовой аналогии.
- ☐ Понимаю, что at-least-once — самый распространённый режим, и он требует **идемпотентности** Consumer'а.
- ☐ Запустил симуляцию ресторана и увидел, как работает очередь с несколькими Consumer'ами.
- ☐ Понимаю связь между нашей симуляцией и реальными технологиями (Redis, Celery, Kafka).

В следующем модуле мы познакомимся с **Redis Streams** — механизмом Append-Only Log в Redis. Мы научимся использовать команды `XADD` и `XREAD` , поймём разницу между классической очередью и логом событий, и подготовимся к работе с Celery.